

The background is dark gray with decorative circuit-like lines in light green and black in the corners. The text is centered in a white, bold, sans-serif font.

welcome to PYTHON 4 PHYSICS!

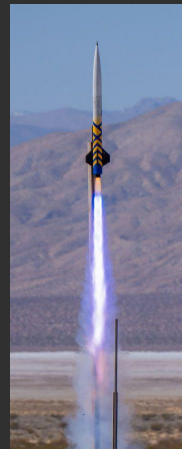
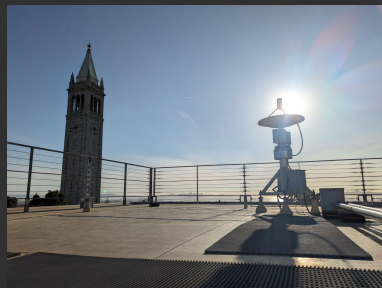
overview



- 1 Introduction
- 2 Lecture:
 - 3 What is Programming?
 - 4 Programming Software
 - 6 Computer Structure
 - 7 Python Basics
- 8 Break
- 9 Live examples
- 10 Project Introduction
- 11
- 12

WHO am I???

- 1 Katie / Katherine Mora
- 2 4th year undergrad @ Berkeley
- 3 Astrophysics + Aerospace Eng.
- 4
- 6 Experience:
- 7 -3rd year teaching coding!
- 8 -Software developer @ SSL for
- 9 NASA Mission Operations
- 10 -Observational Astronomer for 3+ years
- 11 -Career interest in instrumentation
- 12
- 13 For funsies:
- 14 -Love food, love coffee
- 15 -Way too active
- 16 -I like touching grass
- 17 (although its rare)





INTro TO PRoGramming

WHAT IS PROGRAMMING?

1 Programming:

2 Developing instructions that computers can
3 understand and use to execute a specific task

4

6 Computers speak their own language, coding languages are
7 how humans and computers communicate.

8

9 Each coding language has its own strengths.

10

11 Python is a high-level, interpretive programming
12 language!

WHY PYTHON?

1 High level:

2 Easy to understand (for english
3 speakers)

6 Interpretive:

7 Needs an interpreter between code
8 and computer
9 (a.k.a non-compiling code)

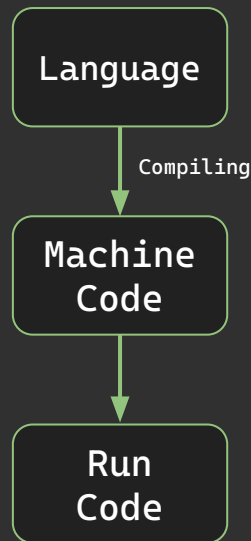
10

11 Compiling code examples: C, C++, ...

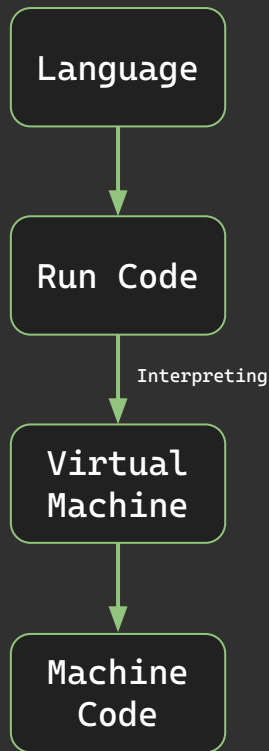
12 Interpreted code: Python, JavaScript, ...

13

Compiled

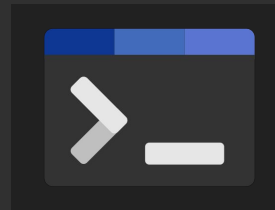


Interpreted



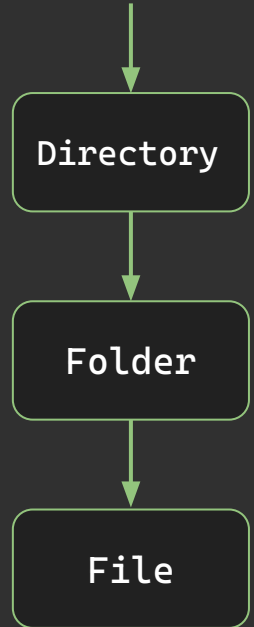
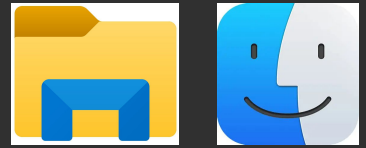
Programming Software

- 1 - Many different coding
- 2 platforms available for free
- 3 - Terminal (all devices), VS
- 4 Code, Bash, Jupyter,
- 6 GitHub, Google Colab, etc.
- 7 - **Google Colab** is the program
- 8 used for this course
- 9 - Available through any
- 10 browser
- 11 - Different programs have
- 12 different strengths



COMPUTER STRUCTURE

- 1 Files are organized in grouped structures
- 2 called directories.
- 3
- 4 Directories, otherwise known as folders, can
- 6 contain more directories.
- 7
- 8 Directory = Folder
- 9
- 10 Directories and folders can contain files too.
- 11
- 12 The default directory is called the home
- 13 directory.



coMpuTer sTRucture

1 Upstream = above or before

2

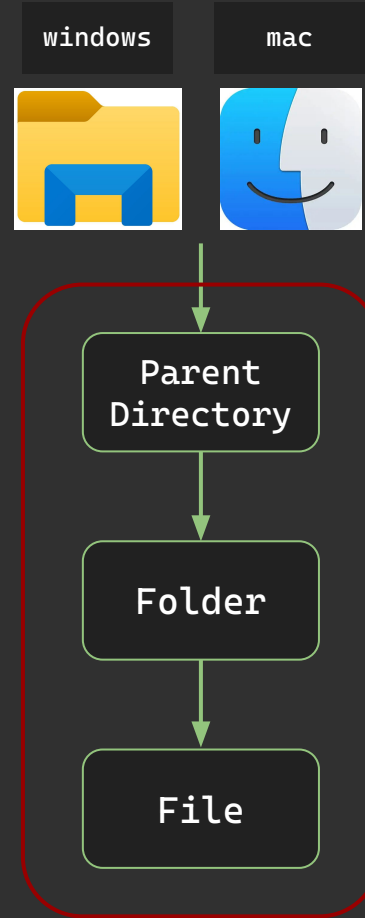
3 Upstream directories are the parent
4 directory to everything held within it.

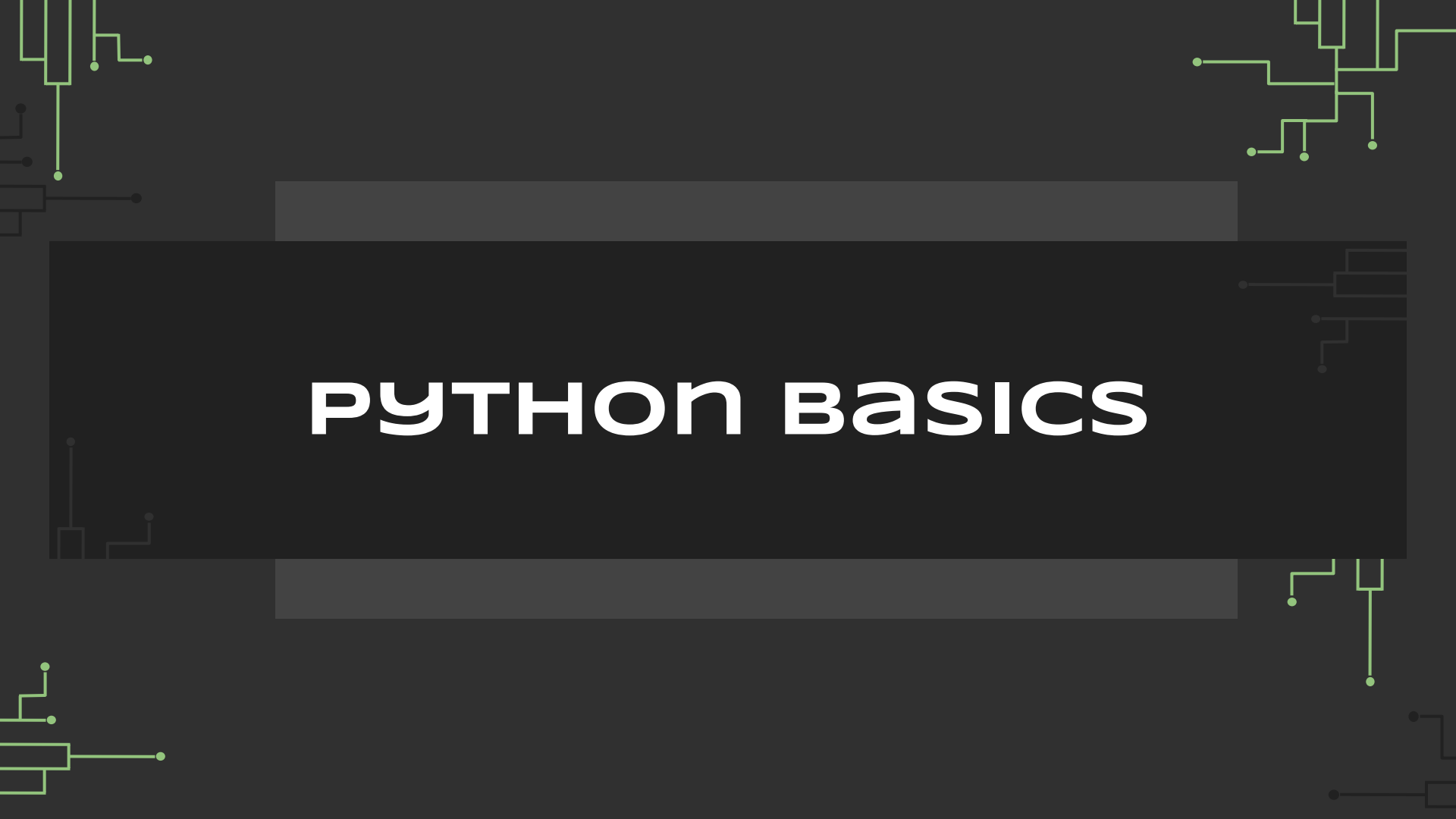
6

7 All the directories and folders above a
8 specific destination is called the
9 **file path**.

10

11 **Important:** A file cannot be accessed without
12 going through ALL the upstream directories.
13 There are NO shortcuts!



The background is a dark gray with abstract, circuit-like patterns in light green and black. These patterns consist of lines, dots, and rectangular shapes, resembling a stylized electronic circuit or data flow. The patterns are distributed across the corners and edges of the frame.

PYTHON BASICS

syntax

1 Syntax:

2 Coding spelling + grammar

3 Overall structure of code

4

6 Syntax is the backbone of coding.

7 If you don't know the syntax, you cannot code.

8

9 Example:

10

11

12

statement to be printed
↓
print("Hello World!")
↑ ↑ ↑
call to string syntax
function

Incorrect
syntax (typos)
is one of the
most common
reasons for
coding errors!

variables

1 To create a variable in python, use the equals = symbol

2

3 `Variable` = holds information in Python

4 Again, very similar to math variables!

6

7 **Example:**

8

```
x = "Hello, World!"
```

9

```
print(x)
```

10

11

12

DATA TYPES

1 Data Types:

2 The classification of a data variable.

3

4 Common Data Types:

6 String, Integer, Float, etc.

7

8 String: words surrounded by “” or ‘’

9 Integer: Any whole number (1, 2, 10, etc.)

10 Float: Any number with a decimal (0.1, .002, 100,000.)

11

12

DATA TYPES VS. DATA STRUCTURES

1 Data structure:

2 Format that can store more than one variable / entry
3 of data

4

6 Data structures contain data types

7

8 Common Data Structure Types:

9 List, Tuple, Dictionary, etc.

10

11

12

LISTS

1 Lists:

2 A collection of items stored in a single variable

3

4 - Ordered

6 - Changeable

7 - Use square brackets:

8 fruits = ["apple", "banana", "cherry"]

9

10

11

12

NOTE ABOUT LISTS

1 Python starts counting from zero! Please keep this in
2 mind!

3

4 So if we have the following list:

6 `my_list = ["chair", "door", "window", "lamp"]`

7

8 Then Python would count the first item at zero:

9 `my_list[0] = "chair", which is the first item`

10 `my_list[1] = "door", which is the second item`

11 `my_list[2] = "window", which is the third item`

12 `my_list[3] = "lamp", which is the fourth item`

↑
index

TUPLE

1 Tuple:

2 A collection of items stored in a single variable

3

4 – Ordered

6 – NOT Changeable

7 – Use parentheses ():

8 weather = ("sunny", "partly cloudy", "rainy")

9

10

11

12

DICTIONARIES

1 Dictionaries:

2 A dictionary can be thought of as a type of list

3 Typical dictionaries look like the one below

4

6

7 names = {"First": "Mia", "Middle": "Marie", "Last": "Gonzales"}

8

9

↑
key

↑
value

↑
key

↑
value

↑
key

↑
value

10

11 Compared to lists, dictionaries are equipped with keys and

12 values.

More on DICTIONARIES

1 We can access values in a dictionary like so:

2 `names["First"]` will return "Mia"

3

4 Attaching `.items()` to the end of a dictionary returns key-value
6 pairs in the form of a tuple

7

8 **Example:**

9

10 `names.items()` returns the following:

11 `(("First", "Mia"), ("Middle", "Marie"), ("Last, Gonzales"))`

12

DATA TYPES example

1 **Example:**

2 What is the data type of the printed statement?

3 `print("Hello World!")`

4

5 **Answer:** String

6

7 Strings *always* have either quotation marks (" ") or

8 apostrophes (' ') surrounding the statement.

FUNCTIONS

1 Functions:

2 Reusable code; acts similar to math functions

3

4 Function Structure:

6

7

8

9

10

11

12

```
def exponent(x, y):  
    z = x ** y  
    return z
```

The diagram illustrates the structure of a Python function definition with the following annotations:

- define function:** A red arrow points to the `def` keyword.
- function name:** A yellow arrow points to the word `exponent`.
- input variables:** Two blue arrows point to the parameters `x` and `y` inside the parentheses.
- indent after define statement:** A green arrow points to the vertical line of indentation before the function body.
- output variables:** A blue arrow points to the variable `z` in the `return` statement.

comments + CODING ETIQUETTE

```
1  Comments are marked with a # before the commented out code:  
2      Everything after the # in the same line of code is  
3      “invisible” when the code is run.
```

```
4  
6  Comments are very helpful for describing your code, and  
7  crucial when collaborating with others.
```

```
8  
9  Example:
```

```
10  
11      def exponent(x, y):  
12          # This function returns x to the y power  
13          z = x ** y  
14          return z
```

comments + CODING ETIQUETTE

1 Good coding etiquette allows for easy to follow code.

2

3 This includes:

4 Using descriptive variable names

6 Commenting logic flow

7 Describing the purpose of functions

8 Consistent syntax / coding style / spacing

9

10 Example:

11 `def exponent(x,y):`

12 `z =x**y`

13 `return z`

14

Better example:

`def exponent(base, exponent):`

`product = base ** exponent`

`return product`



10-minute Break

The background is a dark gray color. It features several abstract, circuit-like line drawings in a light green color. These lines are scattered around the edges of the frame, some forming rectangular shapes and others being more irregular. In the center of the image, there is a large, dark gray rectangular box. Inside this box, the word "Background" is written in a white, bold, sans-serif font. The text is centered both horizontally and vertically within the box.

Background

BASIC OPERATIONS

1 Most mathematical operations are intuitive:

2

3 + Addition

4 - Subtraction

6 * Multiplication

7 / Division

8 = assigns variable

9 Ex: $y = mx + b$

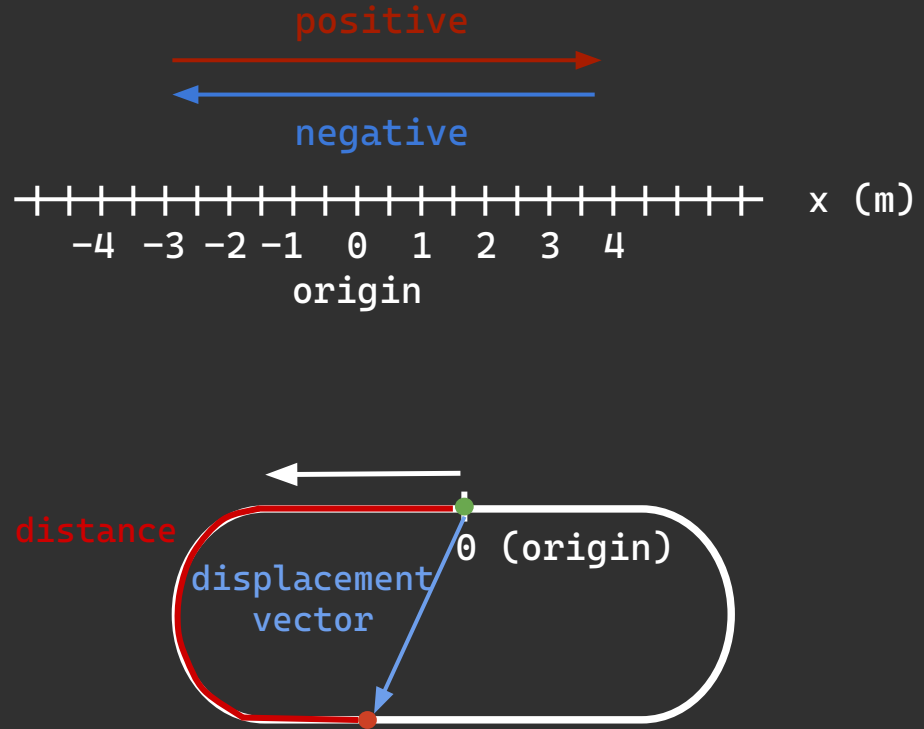
10

11 There are many, many more! See how many you can find :)

12

Background: PHYSICS

- 1 Position:
- 2 location of object
- 3
- 4 Displacement:
- 6 difference vector
- 7 between starting pos.
- 8 and ending pos.
- 9
- 10 Distance:
- 11 total ground covered
- 12



BACKGROUND: PHYSICS equations

1 Useful equations:

2

3 Average Speed = $\frac{\text{total distance travelled}}{\text{time elapsed}} = \frac{d}{t_2 - t_1}$

4 notation for
"average"

6 Average velocity = $\bar{v} = \frac{\text{displacement}}{\text{time elapsed}} = \frac{x_2 - x_1}{t_2 - t_1}$

7

8 Instantaneous velocity = $v = \lim_{\Delta t \rightarrow 0} \frac{\Delta x}{\Delta t} = \frac{dx}{dt}$

9

10

11

12

Background: Projectile motion

2D Kinematics

Constant acceleration

Position as a function of time $x(t) = x_0 + v_0^x t + \frac{a_x}{2} t^2$

$$y(t) = y_0 + v_0^y t + \frac{a_y}{2} t^2$$

Velocity as a function of time $v_x(t) = x'(t) = v_0^x + a_x t$

$$v_y(t) = y'(t) = v_0^y + a_y t$$

Background: Projectile motion

2D Kinematics

Free-fall parabolic motion:

$$y(t) = y_0 + v_0^y t - \frac{g}{2} t^2$$

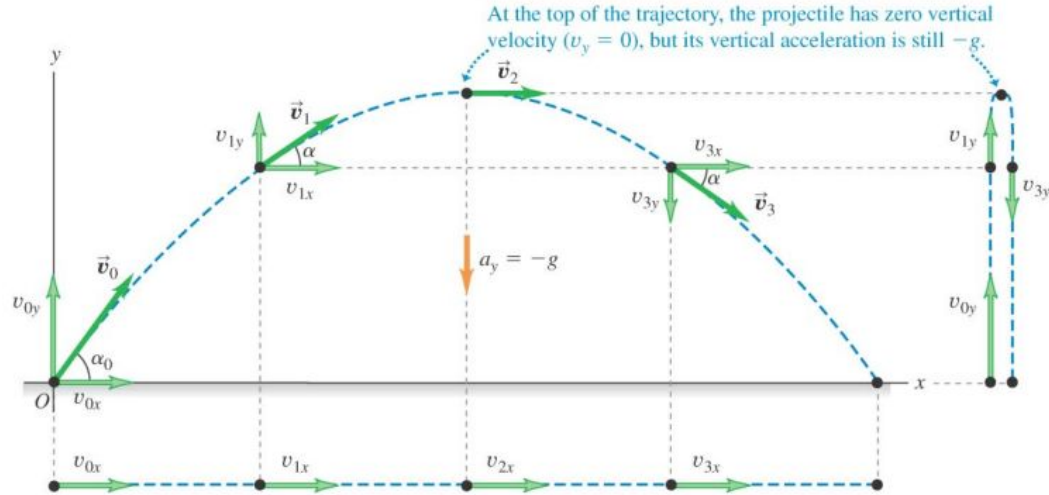
$$x(t) = x_0 + v_0^x t$$

$$g = 9.8 \text{ m/s}^2 = 32.0 \text{ ft/s}^2$$

We can consider x & y motion separately!

BACKGrouND: PROJECTILE motion

Motion in a plane with constant acceleration - Projectile motion



Background: Projectile motion

Projectile motion

Great news!

We can consider x & y motion separately and use the equations we defined in Chapter 2 for motion with constant acceleration

Motion along x -axis: $a_x = 0$

$$v_x = v_{x0}$$

$$x = x_0 + v_{x0}t$$

Motion along y -axis: $a_y = -g$

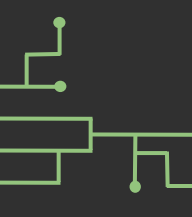
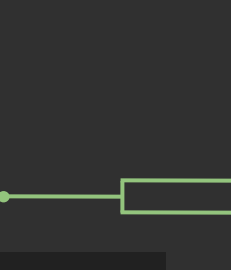
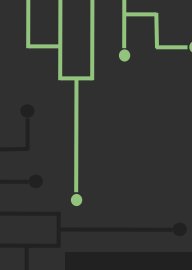
$$v_y = v_{y0} - gt$$

$$y = y_0 + v_{y0}t - \frac{1}{2}gt^2$$

$$v_y^2 = v_{y0}^2 - 2g(y - y_0)$$

$$y = y_0 + \frac{1}{2}(v_{y0} + v_y)t$$

Note, that $a_y = -g = -9.8 \text{ m/s}^2$ only if y -axis is directed upward.



1
2
3
4
6
7
8
9
10

EXAMPLE #1

EXAMPLE #1: equations

1 How would you write the following equations in python?

2

3 Kinematic Velocity Equation (x-direction)

4

6 Kinematic Velocity Equation (y-direction)

7

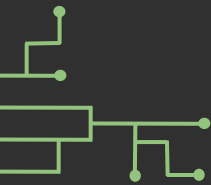
8 Acceleration Equation

9

10

11

12



1
2
3
4
6
7
8
9
10

EXAMPLE #2

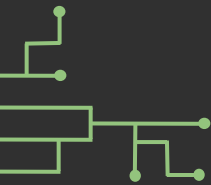
EXAMPLE #2

```
1 See elastic_scattering.ipynb in google drive.  
2  
3  
4  
6  
7  
8  
9  
10  
11  
12
```



1
2
3
4
6
7
8
9
10

PROJECT #1



PROJECT #1: PROJECTILE MOTION

1 Step 1: Write a function for a 1D position as a function
2 of time and initial conditions (for constant
3 acceleration).

4

6 Step 2: Write a function for a 1D velocity as a function
7 of time and initial conditions (for constant
8 acceleration)

9

10 Step 3: Fixing $a_y = 9.8 \text{ m/s}^2$ and $a_x = 0$, write code for
11 plotting the x , y , v_x , v_y .

12

13 Tips: Use `plt.subplot()` to help plot each variable in its own panel / plot. Label the x and y axis using `plt.xlabel()` and `plt.ylabel()` respectively.

PROJECT #1: PROJECTILE MOTION

```
1 Step 4: Write code for plotting y as a function of x for  
2 some initial values of x, y,  $v_x$ ,  $v_y$ .
```

```
3
```

```
4
```

```
6
```

```
7
```

```
8
```

```
9
```

```
10
```

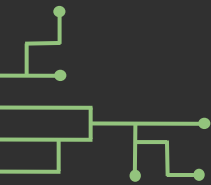
```
11
```

```
12
```



1
2
3
4
6
7
8
9
10

Recap



summary OF TOPICS

- 1 Programming Languages:
- 2 -Compiling and Interpretive
- 3 Programming Software:
- 4 -Terminal, Anaconda, Bash, VS Code, Google Colab,
- 6 Jupyter, etc.
- 7 Computer Structure:
- 8 -Directory / Folder, Files, and File Paths
- 9 Python Basics:
- 10 -Syntax, Data Types (Int, Str, ...), Data Structures
- 11 (List, Tuple, Dictionary)
- 12 Physics Intro: Projectile Motion